Rynox-Style Motorcycle Gear Store: Master Build Prompt

Oct 29, 2026 · @suman

Copy each phase into your AI coding tool one at a time, in order, and finish its acceptance checks before starting the next phase.

Phase 0: Master rules (paste this first, keep it in every session)

Role. You are a senior full-stack engineer and security engineer. Build a complete custom e-commerce website for an Indian motorcycle riding-gear and luggage brand, similar in structure to rynoxgear.com but with original branding, text and images. Do not copy their images, logo or copy. Use a placeholder brand name `YOURBRAND` that I will replace.

Fixed stack (do not change without asking):

Layer	Choice	Why
Frontend	Next.js (App Router) + TypeScript + Tailwind CSS	SSR for SEO, fast mobile pages
Backend	Next.js Route Handlers + service layer (`/src/server`) in the same project	One app to deploy on Hostinger
Database	MySQL 8 or MariaDB (Hostinger managed database)	Included with Hostinger plans
ORM	Prisma with migrations	Typed queries, SQL-injection safe
Auth	Own implementation: argon2id hashing, httpOnly secure session cookies, separate admin sessions	No third-party lock-in
Validation	Zod on every input, client and server	Single source of truth
Images	`sharp` converts uploads to WebP, stored outside the public web root	Speed and safety
Payments	Razorpay (UPI, cards, netbanking, wallets) + Cash on Delivery	Indian checkout

Layer	Choice	Why
Shipping	Shiprocket API (rates, AWB, tracking)	Common in India
Email	Hostinger SMTP via Nodemailer (transactional)	Included with hosting
Hosting	Hostinger (see Phase 9)	As requested

Hostinger check before coding. Node.js apps need a Hostinger plan that supports Node.js web apps (Business or Cloud hosting) or a VPS. Confirm this in hPanel. If the plan is basic shared hosting without Node.js, tell me and switch to a VPS before writing code.

Working rules for you (the AI tool):

- Work phase by phase. At the end of each phase list: files created, how to run it, and a checklist of what I should test.
- Write production-quality code: no placeholder logic, no `TODO` left in security-critical code, no hard-coded secrets. Every secret goes in `.env` and `.env.example` documents every variable.
- Every API route must have: authentication check, role check, Zod validation, rate limit, and a consistent JSON error shape `{ error: { code, message } }`. Never leak stack traces.
- Write tests for money, stock, auth and permission logic.
- Keep UI text in English with a simple structure so Hindi/Bengali can be added later (use a `messages` file, not strings scattered in components).
- Currency is INR, prices stored as integer paise, GST-inclusive display.

Project folder layout:

```
/prisma            schema.prisma, migrations, seed.ts
/src/app           (shop)  public storefront routes
/src/app/admin     admin panel routes (separate layout, separate auth)
/src/app/api       route handlers
/src/server        services, repositories, validators, security helpers
/src/components    ui, shop, admin
/src/lib           config, logger, mail, payments, shipping, rate-limit
/storage           private uploads (outside public web root)
/tests             unit and integration tests
```

Phase 1: Discovery and architecture

Prompt: Before writing any code, produce `ARCHITECTURE.md` and `REQUIREMENTS.md` for the project and wait for my approval. Cover the following.

Business scope. D2C store for motorcycle jackets, pants, gloves, rain gear, luggage, hard parts and accessories, plus a dealer network, a crash-replacement program and an insurance (warranty) registration program.

User roles:

Role	Can do
Guest	Browse, search, add to cart, checkout as guest, track order by order number + phone/email
Customer	Account, saved addresses, wishlist, order history, reviews, claim forms
Dealer applicant	Submits dealer form (India or international), no login
Admin: Super Admin	Everything, including staff management, settings, audit logs
Admin: Catalog Manager	Products, variants, collections, media, size charts
Admin: Order Manager	Orders, shipments, refunds, returns, claims
Admin: Content Manager	Blog, banners, pages, stores/dealers, SEO fields
Admin: Support	Read orders and claims, reply to enquiries, no price or settings access

Modules to design (one section each in `ARCHITECTURE.md`): catalog, inventory, cart, checkout, orders, payments, shipping, customers, reviews, wishlist, coupons, back-in-stock alerts, store and dealer locator, warranty registration, crash-replacement claims, enquiries and dealer leads, blog and CMS pages, banners, SEO, notifications, audit log, admin users and roles, settings.

Request flow to document: Browser → Hostinger edge (SSL, WAF/CDN) → Next.js app (middleware: security headers, rate limit, auth) → service layer (business rules) → Prisma → MySQL. Side channels: Razorpay and Shiprocket webhooks (signature verified) and SMTP.

Non-functional targets: mobile-first, Lighthouse mobile performance 85+, LCP under 2.5 s on 4G, WCAG AA contrast, 99.5% uptime target, daily database backup, all admin actions audited.

Build order (the phases below): design system → database → backend → storefront → admin → integrations → security hardening → deployment → testing and launch.

Acceptance: both documents exist, list assumptions and open questions, and contain an ER overview and a route map. Stop and wait for approval.

Phase 2: Design system and visual style

Prompt: Create the design system first as Tailwind theme tokens plus a `/design` preview page that shows every component. No page may use a color, font size or spacing value that is not a token.

Style direction: bold, rugged, performance-driven adventure-riding look. Dark charcoal surfaces, one electric accent, big condensed uppercase headings, large product photography, sharp 2 to 4 px corners, thin technical dividers, and small certification badges. It should feel like gear for the road, not like a fashion site.

Color tokens (accent is configurable; the default matches the reference look, change it to make your brand unique):

Token	Value	Use
`--bg`	`#0B0C0E`	Page background
`--surface`	`#15171A`	Cards, drawers
`--surface-2`	`#1E2126`	Inputs, hover states
`--line`	`#2B2F36`	Borders, dividers
`--text`	`#F4F5F6`	Main text
`--muted`	`#9AA0A9`	Secondary text
`--accent`	`#C9E943`	Buttons, highlights, active states
`--accent-ink`	`#0B0C0E`	Text on accent buttons
`--danger`	`#FF4D4F`	Errors, sale tags
`--success`	`#3DDC84`	In stock, success

Admin panel uses the same tokens with a light theme option (`--bg: #F6F7F9`) because staff work long hours.

Typography: headings in Barlow Condensed (600 to 800, uppercase, tight tracking); body and UI in Inter. Load both with `next/font` so they are self-hosted (no Google request at runtime). Scale: 12, 14, 16, 18, 24, 32, 44, 64 px; body line height 1.6, headings 1.05.

Core components to build: Button (primary, secondary, ghost, loading), Input, Select, Checkbox, Radio, Quantity stepper, Badge (CE Class A/AA, New, Sale, Restocking Soon), Product card (image swap on hover, color dots, price, rating), Mega menu, Mobile drawer menu, Cart drawer, Quick view modal, Filter drawer, Size chart modal, Review stars, Accordion, Tabs, Toast, Breadcrumbs, Pagination, Skeleton loaders, Empty states, Data table (admin), Form with inline errors, Status chip.

Layout rules:

- Mobile first. Breakpoints 360, 768, 1024, 1280, 1536 px. Product grid: 2 columns on mobile, 3 on tablet, 4 on desktop.
- Header: announcement bar (free shipping text), logo, search, account, wishlist, cart count. Sticky on scroll with reduced height.
- Product page on mobile has a sticky bottom bar with price and Add to Cart.
- Tap targets at least 44 px; visible focus ring in accent color; contrast AA minimum.

- Motion: 150 to 250 ms ease-out, respect `prefers-reduced-motion`.

Page layouts to design (as static mock pages first): Home, Collection, Product, Cart, Checkout, Order confirmation, Account, Dealer locator, Form pages, Blog list, Blog article, 404.

Acceptance: `/design` page renders all components in dark and admin-light themes, passes an accessibility check, and uses tokens only.

Phase 3: Database (MySQL 8 + Prisma)

Prompt: Write `prisma/schema.prisma`, the first migration and `seed.ts` for the tables below. Use `utf8mb4`, InnoDB, `BigInt` or `cuid` ids (never sequential ids exposed in public URLs: use `slug` or a public `orderNumber`), `createdAt` and `updatedAt` on every table, soft delete (`deletedAt`) on catalog and customer tables, and foreign keys with explicit `onDelete` rules. Money is stored as integer paise. Add indexes on every foreign key and every column used in filters or sorting.

Table	Key columns and rules
`users`	id, email (unique), phone, passwordHash (argon2id), name, emailVerifiedAt, failedLogins, lockedUntil, lastLoginAt
`admin_users`	id, email (unique), passwordHash, name, roleId, totpSecret (encrypted), totpEnabled, isActive, lastLoginAt
`roles`, `permissions`, `role_permissions`	Permission keys such as `product.write`, `order.refund`, `settings.write`
`sessions`	id, userId or adminUserId, tokenHash (never store raw token), ip, userAgent, expiresAt, revokedAt
`addresses`	userId, name, phone, line1, line2, city, state, pincode, country, isDefault
`categories`	id, parentId, name, slug (unique), description, imageId, sortOrder, seoTitle, seoDescription, isActive

Table	Key columns and rules
`collections`	id, name, slug, type (manual or rule), rules JSON, bannerId, sortOrder, isActive
`products`	id, name, slug (unique), description, highlights JSON, material, gearType, ceClass (A, AA, none), brandSeries, status (draft, active, archived), gstRate, hsnCode, seoTitle, seoDescription
`product_categories`, `product_collections`	Many-to-many join tables with sortOrder
`product_variants`	id, productId, sku (unique), color, size, pricePaise, comparePricePaise, weightGrams, barcode, isActive
`inventory`	variantId (unique), quantity, reserved, lowStockThreshold, restockEta, allowBackorder
`inventory_movements`	variantId, delta, reason (sale, return, adjust, restock), orderId, adminUserId (stock audit trail)
`media`	id, path (private storage), mimeType, width, height, alt, productId, variantColor, sortOrder
`size_charts`, `size_chart_rows`	Chart per product or category, rows with chest, waist, length, etc.
`carts`, `cart_items`	userId or guestToken, variantId, qty, priceSnapshotPaise, expiresAt
`orders`	id, orderNumber (unique, non-guessable), userId nullable, email, phone, status, paymentStatus, paymentMethod, subtotalPaise, discountPaise, shippingPaise, gstPaise, totalPaise, couponId, shippingAddress JSON, billingAddress JSON, placedAt

Table	Key columns and rules
`order_items`	orderId, variantId, name, sku, qty, unitPricePaise, gstPaise (snapshot of product data at purchase time)
`order_events`	orderId, type, note, actor, createdAt (timeline)
`payments`	orderId, provider, providerOrderId, providerPaymentId, amountPaise, status, rawEvent JSON, verifiedAt
`refunds`	paymentId, orderId, amountPaise, reason, providerRefundId, status
`shipments`	orderId, courier, awb, trackingUrl, status, shippedAt, deliveredAt
`returns`	orderId, itemId, reason, status, pickupAt
`coupons`, `coupon_redemptions`	code (unique), type, value, minOrderPaise, maxUses, perUserLimit, startsAt, endsAt
`reviews`	productId, userId, orderItemId, rating 1 to 5, title, body, status (pending, approved, rejected), verifiedPurchase
`wishlists`	userId, productId (unique pair)
`stock_alerts`	variantId, email, notifiedAt (back-in-stock requests)
`stores`	name, type (exclusive, authorized, online), country, state, city, address, pincode, phone, lat, lng, mapUrl, isActive
`dealer_leads`	type (india or international), name, business, email, phone, location, message, status, assignedTo
`warranty_registrations`	userId nullable, orderNumber, productSku, purchaseDate, name, email, phone, invoiceFileId, status

Table	Key columns and rules
`crash_claims`	claimNumber, warrantyId, incidentDate, description, photos, status, adminNotes, resolution
`enquiries`	name, email, phone, subject, message, status, repliedBy
`blog_posts`	title, slug, excerpt, bodyHtml (sanitized), coverId, authorId, status, publishedAt, seoTitle, seoDescription, tags
`banners`	placement (hero, strip), imageDesktopId, imageMobileId, title, ctaText, ctaUrl, sortOrder, startsAt, endsAt
`pages`	slug (privacy, terms, shipping, returns, wash-care, ce-declarations), title, bodyHtml (sanitized)
`settings`	key, value JSON (store name, GST number, shipping thresholds, free shipping amount)
`audit_logs`	actor, action, entity, entityId, before JSON, after JSON, ip, createdAt
`webhook_events`	provider, eventId (unique), payload, processedAt (idempotency)

Rules the schema and services must enforce:

- Stock is decremented inside a database transaction using a conditional update (`WHERE quantity - reserved >= qty`) so two buyers cannot oversell the last item.
- Order totals are recomputed on the server from variant prices. Never trust prices sent by the browser.
- Webhook events are processed once, using the unique `eventId`.
- Full-text index on product name, description and tags for search.
- Create a least-privilege database user for the app: `SELECT, INSERT, UPDATE, DELETE` only. Migrations run with a separate user that is never used by the running app.

Seed data: one Super Admin (password from an environment variable, forced to change on first login), roles and permissions, 8 categories matching the sitemap (Touring Gear, Offroad Gear, Performance Layers, Luggage, Hard Parts, Accessories, Series collections), 12 sample products with variants and stock, 6 stores, and the legal pages as drafts.

Acceptance: `prisma migrate deploy` and `prisma db seed` run on a clean database, and an ER diagram is exported to `docs/er.md`.

Phase 4: Backend API and business logic

Prompt: Build the backend in `/src/server` as a service layer (business rules) behind thin route handlers in `/src/app/api`. Public routes live under `/api/v1/...`, admin routes under `/api/admin/...` and use a different session cookie and middleware. Every handler follows this pipeline: rate limit → authenticate → authorize (permission key) → Zod-validate params, query and body → call service → return typed JSON. Log each request with a request id (no passwords, tokens or card data in logs).

Public endpoints:

Area	Endpoints
Catalog	`GET /products` (filters: category, collection, size, color, price range, gearType, ceClass, inStock; sort; cursor pagination), `GET /products/:slug`, `GET /collections`, `GET /collections/:slug`, `GET /search?q=` with suggestions
Cart	`GET/POST/PATCH/DELETE /cart` (guest cart by signed token cookie, merged on login)
Checkout	`POST /checkout/quote` (server totals, GST, shipping, coupon), `POST /checkout/pincode` (serviceability and COD availability), `POST /checkout/place`
Payments	`POST /payments/razorpay/create`, `POST /payments/razorpay/verify`, `POST /webhooks/razorpay` (HMAC verified)
Orders	`GET /orders/track` (orderNumber + phone or email), `GET /me/orders`, `POST /me/orders/:id/return`

Area	Endpoints
Auth	`POST /auth/register`, `/login`, `/logout`, `/forgot-password`, `/reset-password`, `/verify-email`, `GET /me`
Customer	addresses CRUD, wishlist, reviews (only for verified purchases)
Forms	`POST /forms/dealer`, `/warranty`, `/crash-claim`, `/enquiry`, `/stock-alert` (CAPTCHA + file upload rules)
Content	`GET /stores`, `GET /blog`, `GET /blog/:slug`, `GET /pages/:slug`, `GET /banners`

Admin endpoints (all require admin session and a permission key): dashboard stats, CRUD for products, variants, inventory adjustments, categories, collections, media upload, size charts, coupons, banners, pages, blog, stores, orders (status changes, notes, shipment creation, refunds), returns, reviews moderation, warranty and claim handling, dealer leads, enquiries, customers (read, block), staff users and roles, settings, audit log viewer, CSV exports.

Business rules:

1. **Pricing:** server computes subtotal, discount, GST (inclusive display, breakup on invoice: CGST + SGST within state, IGST otherwise) and shipping. Free shipping above the amount in `settings`. COD fee and COD limit are settings.
2. **Inventory:** reserve stock at checkout start for 15 minutes, release on expiry (scheduled job), commit on payment success, restore on cancel or refund.
3. **Order status flow:** `pending_payment → paid → processing → shipped → delivered`, with `cancelled`, `returned`, `refunded` branches. Only allowed transitions are accepted; each writes an `order_events` row.
4. **Payment verification:** never mark an order paid from the browser redirect. Mark paid only after server-side signature verification or a verified webhook; make it idempotent.
5. **Coupons:** validate dates, minimum order, per-user and total limits inside the order transaction.
6. **Reviews:** only customers with a delivered order item; moderated before publishing.
7. **Search:** MySQL full-text first; keep a repository interface so it can be replaced later.

8. **Scheduled jobs** (cron route protected by secret, or Hostinger cron): release expired reservations, abandoned-cart email after 1 hour and 24 hours, back-in-stock notifications, delete expired sessions, nightly low-stock report to admins.
9. **Invoices:** generate GST invoice PDF per order with invoice number series, store privately, link in customer account.
10. **Uploads:** accept JPG, PNG, WebP, PDF only; check magic bytes (not just extension); max 5 MB per image and 8 MB per PDF; random file names; scan with size and dimension limits; store outside public root and serve through an authenticated or signed route.

Error handling: one `AppError` class with codes (`VALIDATION`, `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `CONFLICT`, `RATE_LIMITED`, `PAYMENT_FAILED`); a global handler maps them to HTTP status codes.

Tests required: pricing and GST, stock race condition, coupon limits, order state machine, webhook idempotency, role and permission matrix, IDOR checks (a customer cannot read another customer's order).

Acceptance: an OpenAPI file (`docs/openapi.yaml`) lists every endpoint, and all tests pass.

Phase 5: Storefront frontend

Prompt: Build the public site using the Phase 2 design system and the Phase 4 API. Use server components for SEO pages, client components only where interaction is needed. Every page needs loading skeletons, error and empty states, and works on a 360 px phone.

Pages and what each must contain:

Page	Route	Required content and behavior
Home	`/`	Announcement bar; hero slider (banners from admin, desktop and mobile images, first image preloaded); category shortcut tiles; curated collection rows (Monsoon Ready, Evo, GT, Pro, Luggage) with horizontal scroll on mobile; trust strip (CE certified, free shipping, crash replacement, easy returns); new arrivals; blog teasers; newsletter signup; footer with links and payment icons (UPI, cards, COD)
Collection	`/collections/[slug]`	Title, description, product count; filter drawer (size, color, price slider, gear type, CE class, availability); sort; grid with cursor pagination or Load more; filters stored in the URL so pages can be shared
Product	`/products/[slug]`	Image gallery with zoom and swipe; color swatches that swap images; size selector with out-of-stock sizes disabled; "Restocking Soon" label with notify-me form; price, compare price, GST-inclusive note; pincode delivery and COD check; size chart modal; CE badge and warranty messaging; tabs (details, materials, care, shipping, returns); reviews with stars and photos; related products; sticky mobile Add to Cart bar; Product JSON-LD

Page	Route	Required content and behavior
Search	`/search`	Live suggestions after 2 characters, result page with the same filters as collections
Cart	drawer + `/cart`	Line items, quantity stepper, stock warnings, coupon field, shipping estimate, order summary
Checkout	`/checkout`	Contact, address with pincode auto-fill of city and state, shipping method, payment choice (UPI, card, netbanking, COD), order summary; guest checkout allowed; fields validate inline; double-submit protection
Order success / failed	`/orders/[number]`	Status timeline, invoice download, tracking link
Track order	`/track-order`	Order number + phone or email
Account	`/account/...`	Login, register, forgot password, profile, addresses, orders, wishlist, warranty registrations, claims
Store locator	`/stores`	Tabs: Exclusive stores, Authorized dealers (India), International dealers, Online resellers; search by state or city; map links
Forms	`/dealer-india`, `/dealer-international`, `/warranty`, `/crash-replacement`, `/contact`	Accessible forms, client + server validation, CAPTCHA, success message, file upload for invoice and crash photos
Blog	`/blog`, `/blog/[slug]`	List with tags, article with table of contents, Article JSON-LD

Page	Route	Required content and behavior
Legal and info	`/pages/[slug]`	Privacy, Terms, Shipping, Returns, Wash & Care, CE Declarations
404 / 500		Branded pages with search and links

Frontend technical requirements:

- Images: `next/image`, WebP/AVIF, explicit width and height, lazy loading except the hero image.
- State: cart in server-backed storage with optimistic UI; no sensitive data in `localStorage`.
- Forms: React Hook Form + the same Zod schemas as the server.
- SEO: unique `<title>` and meta description from admin fields, canonical URLs, Open Graph, `sitemap.xml`, `robots.txt`, Product, Breadcrumb, Organization and Article structured data, clean URLs, `noindex` on cart, checkout and account pages.
- Analytics: GA4 and Meta Pixel loaded only after cookie consent; fire `view_item`, `add_to_cart`, `begin_checkout`, `purchase`.
- Accessibility: semantic HTML, keyboard-friendly menus and drawers with focus trap, alt text, labels on every input.
- Security in the browser: never use `dangerouslySetInnerHTML` except for sanitized CMS content; no secrets in client bundles (only `NEXT_PUBLIC_` values that are safe to expose).
- Cookie consent banner and a clear privacy link.

Fix the reference-site issues: show UPI, cards and COD (not only PayPal) in the footer and checkout; every top menu item must have its own collection page (for example Offroad Gear); show back-in-stock alerts on every sold-out variant.

Acceptance: all routes render with seed data, Lighthouse mobile is 85 or higher on Home, Collection and Product, and a full purchase works in Razorpay test mode and COD.

Phase 6: Admin panel

Prompt: Build the admin panel at `/admin` as a separate layout with its own login page (`/admin/login`), its own session cookie, and a sidebar navigation. Use the light theme by default with a dark toggle. Every screen is permission-gated both in the UI (hide) and on the server (enforce). Use data tables with search, filters, sorting, pagination, column visibility and CSV export.

Security of the admin area:

- Separate `admin_users` table and session; admin login is rate limited (5 attempts per 15 minutes per IP and per account) with account lockout and a generic error message.
- Mandatory two-factor authentication (TOTP app) for every admin; show QR on first login and 8 one-time recovery codes.
- Session idle timeout 30 minutes, absolute timeout 8 hours, re-enter password for sensitive actions (refund, delete, role change, settings).
- Optional IP allowlist setting for `/admin`; `X-Robots-Tag: noindex` and no links to admin from the storefront.
- Every create, update and delete writes to `audit_logs` with before and after values.
- CSRF token on every state-changing admin request.

Screens:

Section	Screens and features
Dashboard	Today's and 30-day revenue, orders by status, average order value, conversion proxy, top products, low-stock list, pending claims, pending reviews, pending dealer leads; date range filter
Products	List with image, SKU, stock, status; create and edit form with tabs (General, Variants, Pricing and GST, Inventory, Images, Size chart, SEO, Related); variant matrix generator (colors × sizes); drag to reorder images; bulk edit price and status; duplicate product; import and export CSV; archive and restore

Section	Screens and features
Inventory	Stock per variant, quick adjust with reason, movement history, low-stock threshold, restock ETA (drives the "Restocking Soon" label), stock-alert requests count
Categories and Collections	Tree editor with drag and drop, banner, SEO, rule-based or manual collections
Orders	List with filters (status, payment, date, method); order detail with items, addresses, payments, timeline; actions: confirm COD, mark packed, create shipment (Shiprocket), print label and invoice, cancel, partial or full refund, add internal note, resend email
Returns and Refunds	Queue with approve or reject, pickup scheduling, refund to original method
Customers	List, detail with orders and addresses, block or unblock, export; no password viewing
Reviews	Moderate (approve, reject, reply), filter by rating
Warranty and Crash Claims	Registration list with invoice preview; claim workflow (new → in review → approved or rejected → resolved), notes, attachments, status email to customer
Dealers and Stores	CRUD for stores and dealers with map URL and coordinates; dealer leads with status and assignee
Enquiries	Inbox with status and reply template
Marketing	Coupons, banners (schedule start and end), email templates, abandoned cart settings, newsletter subscribers export
Content	Blog posts with rich text editor (sanitized HTML), legal and info pages, homepage section order, menu editor (mega menu)
SEO	Default meta templates, redirects manager (301), sitemap refresh

Section	Screens and features
Staff and Roles	Create staff, assign role, reset 2FA, deactivate; role editor with permission checkboxes
Settings	Store details, GST number, invoice prefix, shipping rules and free-shipping threshold, COD limit and fee, payment keys status (masked), email templates, maintenance mode
Logs	Audit log with filters, failed logins, webhook events, background job runs

UX details: unsaved-changes warning, confirmation modal for destructive actions, toast feedback, keyboard shortcuts for search, skeleton loading, responsive layout for tablets.

Acceptance: a Catalog Manager cannot open Orders, a Support user cannot change prices, all actions appear in the audit log, and a created product immediately appears on the storefront.

Phase 7: Payments, shipping and messaging integrations

Prompt: Implement each integration behind an interface (`PaymentProvider` , `ShippingProvider` , `Mailer` , `Notifier`) so providers can be swapped. Keep all keys in environment variables, support test and live modes, and write a mock provider for local development.

Razorpay (payments):

1. Server creates the Razorpay order with the server-computed amount; the browser opens Razorpay Checkout with that order id only.
2. After payment, the browser sends `razorpay_order_id` , `razorpay_payment_id` and `razorpay_signature` to `/payments/razorpay/verify` ; the server verifies the HMAC-SHA256 signature with the key secret using a constant-time comparison.
3. `/webhooks/razorpay` is the source of truth: verify `X-Razorpay-Signature` against the raw request body, store the event id for idempotency, and handle `payment.captured` , `payment.failed` , `refund.processed` .
4. Refunds from the admin call the Razorpay refund API and update `refunds` only after confirmation.
5. Store no card data. Only provider ids and status are saved.

Cash on Delivery: available only for serviceable pincodes under the COD limit; optional OTP or phone confirmation before dispatch to reduce fake orders; COD fee from settings; mark paid when the courier reports collection.

Shiprocket (shipping): authenticate with a token refreshed automatically; pincode serviceability and rate lookup at checkout; create order and AWB from the admin; store label and tracking URL; webhook or scheduled poll updates shipment status to shipped, out for delivery, delivered, RTO; customer sees live status.

Email (Nodemailer + Hostinger SMTP): templates for welcome, verify email, password reset, order confirmation with invoice, shipped, delivered, refund, abandoned cart, back-in-stock, claim status, dealer lead acknowledgement, and admin alerts. Queue emails in a table and send from a job so checkout never waits on SMTP. Add SPF, DKIM and DMARC records for the domain.

WhatsApp / SMS (optional, behind a feature flag): order confirmation and shipping updates through an approved provider API with opt-in stored per customer.

Other integrations: Google reCAPTCHA v3 or Cloudflare Turnstile on forms and login; Google Analytics 4 and Meta Pixel after consent; Google Search Console verification file.

Acceptance: in test mode, a card payment, UPI payment, failed payment, COD order, refund and shipment creation all update orders correctly, and replaying the same webhook twice changes nothing.

Phase 8: Security hardening (check everything)

Prompt: Act as a security auditor on the code you built. Implement every control below, then write `SECURITY.md` that maps each control to the file where it lives, and produce a pass/fail report for the checklist. Fix every failure before moving on. Follow the OWASP Top 10 and OWASP ASVS Level 2 as the reference.

1. Authentication and sessions

- Passwords hashed with argon2id (memory 64 MB, 3 iterations); minimum 10 characters; check against a common-password list; never log or email passwords.
- Session tokens are random 32 bytes; only the SHA-256 hash is stored; cookies are `HttpOnly; Secure; SameSite=Lax` (admin: `Strict`) with the `__Host-` prefix; rotate the session on login and on privilege change; revoke all sessions on password reset.

- Login, register, forgot-password and OTP endpoints are rate limited per IP and per account; lock the account for 15 minutes after 5 failures; return identical messages for unknown email and wrong password.
- Password reset tokens are single use, hashed in the database and expire in 30 minutes.
- Admin accounts require TOTP two-factor; encrypt the TOTP secret at rest (AES-256-GCM with a key from the environment).

2. Authorization

- Deny by default. Permission check inside every service method, not only in middleware.
- Prevent IDOR: every query for customer data includes the owner id (`WHERE id = ? AND userId = ?`). Order tracking requires order number plus phone or email.
- Test the role matrix automatically; add a test for each admin permission.

3. Input, injection and output

- Zod validation on every request with strict schemas (reject unknown fields), size limits on bodies (1 MB JSON, uploads separate).
- Only Prisma queries or parameterized SQL; if raw SQL is needed, use `$queryRaw` with tagged templates, never string concatenation.
- Sanitize CMS and blog HTML with an allowlist sanitizer (DOMPurify on the server) before saving and again before rendering.
- Escape all output; no `eval`, no `new Function`, no `dangerouslySetInnerHTML` on user input.
- Block SSRF: the server never fetches user-supplied URLs.
- Prevent open redirects: `returnTo` values must be relative paths on an allowlist.

4. HTTP security headers (middleware, all routes)

Header	Value
`Strict-Transport-Security`	`max-age=63072000; includeSubDomains; preload`

Header	Value
`Content-Security-Policy`	`default-src 'self'; script-src 'self' 'nonce-<random>' https://checkout.razorpay.com https://www.googletagmanager.com; frame-src https://api.razorpay.com; img-src 'self' data: blob: https:; connect-src 'self' https://lumberjack.razorpay.com https://www.google-analytics.com; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'` (adjust after testing each third party)
`X-Content-Type-Options`	`nosniff`
`X-Frame-Options`	`DENY`
`Referrer-Policy`	`strict-origin-when-cross-origin`
`Permissions-Policy`	`camera=(), microphone=(), geolocation=(), payment=(self)`
`Cross-Origin-Opener-Policy`	`same-origin`

Remove the `X-Powered-By` header. Cache `private, no-store` on account, cart, checkout and admin responses.

5. CSRF and CORS

- Double-submit or synchronizer token on all state-changing requests; check `Origin` and `Host` headers; SameSite cookies as a second layer.
- CORS disabled by default; allow only your own domains if an external client is ever needed. Never use `*` with credentials.

6. File uploads

- Allowlist by magic bytes and MIME, size and pixel limits, random names, re-encode images with `sharp` (this strips scripts and EXIF), store outside the public directory, serve via a route that checks permission; never execute uploaded files; no SVG uploads from the public.

7. Payments and webhooks

- Server-side price calculation only; HMAC signature checks with constant-time comparison; idempotent webhooks; no card data stored or logged (PCI scope stays at SAQ A because Razorpay hosts the card fields); fraud checks such as velocity limits per IP and phone, COD confirmation.

8. Secrets and configuration

- All secrets only in environment variables set in hPanel, never committed; `.env` in `.gitignore`; run a secret scanner (gitleaks) in CI; separate keys for test and live; rotate the keys at launch; `.env.example` has dummy values.
- Validate environment variables on startup with Zod and refuse to boot if any is missing.

9. Dependencies and build

- Commit the lockfile, pin versions, run `npm audit --omit=dev` and fix high or critical findings; enable Dependabot or Renovate; remove unused packages; build with `NODE_ENV=production`; disable source maps in production.

10. Rate limiting and abuse protection

Target	Limit
Login and admin login	5 per 15 minutes per IP and per account
Register, forgot password	5 per hour per IP
Forms (dealer, warranty, enquiry)	5 per hour per IP, plus CAPTCHA
Search and product API	120 per minute per IP
Checkout and payment endpoints	20 per 10 minutes per IP

Keep counters in MySQL or Redis, not only in process memory (the app restarts and may run several processes). Block known bad bots in robots and at the edge.

11. Logging, monitoring and privacy

- Structured logs with request id; never log tokens, passwords, OTPs, full card or full personal data; keep 90 days.
- Alert by email on repeated failed admin logins, webhook signature failures, 5xx spikes and low disk.

- Privacy: cookie consent, privacy policy, data export and delete-account request flow, retention rules, data minimization. Follow India's Digital Personal Data Protection Act, 2023 requirements for consent and data principal rights, and keep GST invoices for the legally required period.

12. Database

- App user has no `DROP`, `ALTER`, `GRANT` or `FILE` rights; MySQL not exposed to the internet (use localhost or an allowlisted IP for remote access); encrypt sensitive columns (TOTP secrets, tokens); daily backups tested by a real restore.

Security acceptance tests (automated where possible):

1. SQL injection strings in every input do not change results.
2. Stored and reflected XSS payloads render as text.
3. A customer cannot fetch another customer's order, address or invoice by changing an id.
4. A Support admin gets 403 on price update, refund and settings routes.
5. State-changing requests without a CSRF token fail.
6. Tampered Razorpay signature and tampered price are rejected.
7. Uploading a `.php`, `.html` or a renamed executable is rejected.
8. Headers scan shows grade A on securityheaders.com and SSL Labs shows A or better.
9. `npm audit` shows no high or critical issues; gitleaks finds no secrets.

Acceptance: `SECURITY.md` lists all controls with file references, and the nine tests above pass.

Phase 9: Hostinger deployment

Prompt: Prepare the project for Hostinger and write `DEPLOY.md` with exact click-by-click steps for me (I am not a DevOps expert). Support two paths and tell me which one my plan needs: **Path A** Hostinger Node.js web app hosting (Business or Cloud plan), **Path B** Hostinger VPS (Ubuntu). Hostinger menus and plan limits change, so double-check the current hPanel screens before giving steps.

Code changes needed for Hostinger:

- Next.js `output: 'standalone'`, `poweredByHeader: false`, production source maps off, health route `GET /api/health` (checks the database, returns no secrets).

- `package.json` scripts: `build`, `start`, `prisma:deploy`, `seed`, `lint`, `test`. Node version pinned in `engines` and `.nvmrc` (Node 20 or 22 LTS, whichever Hostinger lists).
- All file paths configurable by environment variables (`STORAGE_DIR`, `INVOICE_DIR`) so private files live outside the deployed app folder and survive redeploys.
- A storage interface with two drivers: local disk and S3-compatible storage (Cloudflare R2, Backblaze or AWS S3). Recommend S3-compatible storage for production so uploads are never lost on redeploy and can be served through a CDN.
- Trust proxy settings and correct client IP reading for rate limits behind Hostinger's proxy.

Path A: Hostinger Node.js app (steps to document):

1. Buy the plan and domain; point DNS to Hostinger; wait for propagation.
2. In hPanel create a MySQL database, a database user with a strong generated password, and assign only that database. Note the host, name and user.
3. Create a second database user for migrations (higher rights) used only during deployment.
4. Push the code to a private GitHub repository (no secrets inside).
5. In hPanel add a Node.js web app, connect the GitHub repository (or upload a build archive), choose the Node version, set build command (`npm ci && npx prisma generate && npm run build`) and start command (`npm run start`).
6. Add environment variables in hPanel: `DATABASE_URL`, `SESSION_SECRET`, `ENCRYPTION_KEY`, `RAZORPAY_KEY_ID`, `RAZORPAY_KEY_SECRET`, `RAZORPAY_WEBHOOK_SECRET`, `SHIPROCKET_EMAIL`, `SHIPROCKET_PASSWORD`, `SMTP_HOST`, `SMTP_USER`, `SMTP_PASS`, `NEXT_PUBLIC_SITE_URL`, `CRON_SECRET`, `STORAGE_DIR`, CAPTCHA keys. Generate secrets with `openssl rand -base64 48`.
7. Run migrations once (`npx prisma migrate deploy`) using the migration user, then seed, then switch the app to the restricted database user.
8. Install the free SSL certificate, enable Force HTTPS, and redirect `www` to the main domain (or the reverse) with a single 301.
9. Add cron jobs in hPanel that call `/api/cron/<job>` with the `CRON_SECRET` header every 5 to 15 minutes.
10. Set Razorpay and Shiprocket webhook URLs to `https://yourdomain.com/api/webhooks/...` and confirm signature checks.
11. Create the first admin, enable 2FA, change the seeded password, then delete the seed admin env variable.

Path B: VPS (if Node.js hosting is not available):

- Ubuntu LTS; create a non-root sudo user; disable root SSH login and password login (SSH keys only); change SSH port optionally; enable UFW allowing only 22, 80, 443; install fail2ban; enable unattended security upgrades.
- Install Node LTS, MySQL (bound to `127.0.0.1`), Nginx as reverse proxy, PM2 to keep the app running and restart after reboot; Certbot for free SSL with auto-renewal.
- Nginx: gzip or brotli, cache `/_next/static` for 1 year, `client_max_body_size 10m`, rate limiting zone for login routes, security headers repeated at the edge, hide server tokens.
- Run the app as an unprivileged user; app folder not writable by the web server except the storage folder.

Both paths:

Task	Requirement
Backups	Daily `mysqldump` (compressed, encrypted) to storage off the server, 30 days retention; weekly full file backup; test a restore before launch and monthly after
Staging	Create a staging subdomain with its own database and Razorpay test keys; deploy there first
CDN and firewall	Use Cloudflare (free plan) in front of the domain for CDN, DDoS protection, bot rules and a WAF; set SSL mode Full (strict)
Email deliverability	SPF, DKIM, DMARC records; send a test to Gmail and Outlook
Monitoring	Uptime monitor on `/api/health` every 5 minutes (UptimeRobot or Better Stack); error tracking with Sentry (scrub personal data); disk and memory alerts
Deploy process	Git tag → build → migrate → restart; keep the last release to roll back in minutes
Maintenance	Maintenance-mode setting in admin; monthly dependency update

Acceptance: the site runs on the live domain over HTTPS, `/api/health` is green, a backup restore has been tested on staging, and `DEPLOY.md` explains rollback in under ten lines.

Phase 10: Testing, performance, SEO and launch

Prompt: Run a full pre-launch review and produce `LAUNCH_REPORT.md` with pass or fail for each item. Fix all failures, then give me a final go-live checklist.

Functional testing

- [ ] Guest purchase with UPI, card (test cards), netbanking and COD
- [ ] Failed and abandoned payment returns stock and leaves a recoverable cart
- [ ] Registered customer: sign up, verify email, reset password, saved address, reorder, wishlist, review
- [ ] Every variant shows the right price, image and stock; "Restocking Soon" and notify-me work and send email when stock returns
- [ ] Coupon rules (minimum order, expiry, per-user limit) behave correctly
- [ ] GST invoice shows correct CGST, SGST or IGST split and totals
- [ ] Pincode check, shipping rate and COD limit work for metro and remote pincodes
- [ ] Return, refund and crash-replacement claim flows end to end
- [ ] Dealer forms, warranty form and contact form reach the admin inbox and send email
- [ ] Admin: every role sees only what it should; audit log records changes

Compatibility

- [ ] iPhone Safari, Android Chrome (low-end device), desktop Chrome, Firefox, Safari, Edge
- [ ] Screen widths 360, 390, 768, 1024, 1440; landscape phone; 200% zoom
- [ ] Keyboard-only navigation and screen reader pass on checkout

Performance targets

Metric	Target (mobile, 4G)
Largest Contentful Paint	under 2.5 s
Interaction to Next Paint	under 200 ms
Cumulative Layout Shift	under 0.1

Metric	Target (mobile, 4G)
JavaScript on home page	under 200 KB compressed
Hero image	under 150 KB, WebP or AVIF

Also: database indexes verified with `EXPLAIN` on collection, search and order queries; cache public pages with ISR and revalidate when admin edits; Gzip or Brotli on; load test the checkout with 50 concurrent users.

SEO

- [ ] Unique title and description per page, canonical tags, one `h1` per page
- [ ] `sitemap.xml` and `robots.txt` live; sitemap submitted to Google Search Console
- [ ] Product, Breadcrumb, Organization, Article and FAQ structured data validated
- [ ] 301 redirects from any old URLs; no broken links; alt text on all images
- [ ] Open Graph and Twitter cards; Google Merchant Center product feed (optional)
- [ ] Content: 10 starter blog posts targeting riding-gear searches (monsoon gear, touring jackets, CE ratings, luggage guides)

Legal and compliance

- [ ] Privacy Policy, Terms, Shipping, Returns and Refund, Cookie policy published
- [ ] GST number and business address in footer and invoices; Grievance officer contact as required for Indian e-commerce
- [ ] Cookie consent working; analytics fire only after consent
- [ ] CE declarations of conformity uploaded as PDFs

Go-live steps

1. Switch Razorpay and Shiprocket to live keys and rotate all secrets.
2. Run one real low-value order and one refund on production.
3. Confirm backups ran and a restore works.
4. Turn on uptime monitoring and error alerts; give a second person admin access with 2FA.
5. Set DNS TTL low before cutover, switch DNS, verify HTTPS and redirects.
6. Submit sitemap, verify Search Console and GA4 events.
7. Remove seed and test data, test admins and the `/design` page (disable in production).

First 30 days: watch errors, slow queries, abandoned carts, search terms with no results and payment failure rate; patch dependencies weekly; review the audit log and failed-login report weekly; plan phase two (wishlist sharing, loyalty points, Hindi language, WhatsApp order updates, product bundles).

Acceptance: `LAUNCH_REPORT.md` shows every item passed or has a written waiver, and I have the live site, repository, `.env.example`, `SECURITY.md`, `DEPLOY.md` and the admin user guide.